

1: 更新单个对象字段（map + 浅拷贝）

题干：

你有一个用户列表，点击“改名”后，把 `id=2` 的用户 `name` 改成 `"Bobby"`。

要求：

1) 返回**新数组**；2) 只为被修改的那一项创建**新对象**（浅拷贝）；3) 其他项尽量复用原对象引用（性能更好）。

```
const users = [
  { id: 1, name: "Alice", age: 18 },
  { id: 2, name: "Bob", age: 19 },
  { id: 3, name: "Cindy", age: 20 },
];
const targetId = 2;
```

2: 切换 todo 的 done（map + 浅拷贝）

题干：

点击某个 todo，需要把它的 `done` 状态取反（`true ↔ false`）。

要求同上：不改原数据，只改目标项。

```
const todos = [
  { id: 1, text: "learn map", done: false },
  { id: 2, text: "learn react", done: true },
];
const targetId = 1;
```

3: 删除一项（filter，不需要拷贝对象但需要新数组）

题干：

删除 `id=3` 的商品。

要求：返回新数组，不改原数组。

```
const products = [
  { id: 1, name: "Book" },
  { id: 2, name: "Pen" },
  { id: 3, name: "Cup" },
];
const removeId = 3;
```

4：批量“标记为已读”（map + 浅拷贝）

题干：

通知列表里，把 `unread: true` 的项都标记为已读（`unread → false`）。

要求：只对被修改的项创建新对象。

```
const notifs = [
  { id: 1, title: "A", unread: true },
  { id: 2, title: "B", unread: false },
  { id: 3, title: "C", unread: true },
];
```

5：更新“嵌套对象”的一个字段（需要两层浅拷贝）

题干：

用户信息结构如下，点击后要把 `id=1` 的用户的 `profile.city` 改成 `"Shanghai"`。

注意：`profile` 是嵌套对象。

要求：必须做到：

- `users` 数组新引用
- 目标 `user` 对象新引用
- 目标 `user` 的 `profile` 对象也新引用
（否则会出现“看似没改，但其实改了原对象”或 `React` 依赖比较出错）

```
const users = [
  { id: 1, name: "Alice", profile: { city: "Beijing", level: 3 } },
  { id: 2, name: "Bob", profile: { city: "Tianjin", level: 2 } },
];
```

6：购物车：增加数量或新增商品（reduce + 浅拷贝）

题干：

实现一个纯函数 `addToCart(cart, item)`：

- 如果 `cart` 里已有同 `id` 商品：把它的 `count + 1`
 - 没有：追加一条 `{...item, count: 1}`
- 要求：不改原 `cart`、不改原 `item`；返回新数组。

```
const cart = [
  { id: 1, name: "Apple", count: 2 },
  { id: 2, name: "Milk", count: 1 },
];

const item = { id: 2, name: "Milk" }; // 这次要加 Milk
```

7：撤销删除：用 reduce 生成新结构（列表 + 回收站）

题干：

你有文件列表 `files`，删除时并不是直接丢掉，而是分到：

- `active`：未删除
- `trash`：已删除（带上 `deletedAt` 时间戳字段）

要求：使用 `reduce` 返回一个对象 `{ active, trash }`，并确保被丢进 `trash` 的对象是“浅拷贝出的新对象”（加 `deletedAt`），不污染原对象。

```
const files = [
  { id: 1, name: "a.png", deleted: false },
  { id: 2, name: "b.png", deleted: true },
  { id: 3, name: "c.png", deleted: false },
];
```

8：编辑表单：批量更新多个字段（Object.assign + 浅拷贝）

题干：

你在 React 里有一个 `form` 状态对象：

```
const form = { name: "Jett", email: "a@a.com", age: 18 };  
const patch = { email: "b@b.com", age: 19 };
```

现在要把 `patch` 合并到 `form`，返回新对象。要求：

1) 不能修改原 `form`；2) 用 `Object.assign` 写出答案（强调浅拷贝）。

9：根据 action 更新 state（Redux/Reducer 风格，map + 多层浅拷贝）

题干：

给你一个 `state`（非常像 `reducer`）：

```
const state = {  
  selectedId: 2,  
  items: [  
    { id: 1, title: "A", meta: { pinned: false } },  
    { id: 2, title: "B", meta: { pinned: false } },  
  ]  
};
```

提醒：`meta` 是嵌套对象。

当执行“pin 当前选中项”时：

- 把 `selectedId` 对应 item 的 `meta.pinned` 设为 `true`
- 返回全新的 `state`（`state` 对象、`items` 数组、目标 item 对象、目标 `meta` 对象都要新引用）

10：性能意识：只在需要时创建新对象（map + 条件浅拷贝）

题干：

你有 1000 条消息，想把某个用户 `userId=9` 的所有消息 `isMuted` 设为 true。

要求：

- 必须返回新数组
- 但只有那些“需要变更的消息”才创建新对象；其他消息必须复用旧对象引用
(React 性能优化：减少无意义的重新渲染)